

Universal-Input Proof Progress for the FreeRTOS V6.1.1 Scheduler

Mathematical progress report - blind Isabelle/HOL reconstruction

Status date: 2 August 2026 | Result: reusable universal-input bricks checked; universal functional correctness remains open

Scope warning. This manuscript does **not** claim universal correctness of `vTaskDelay`, the five scheduler roots, or the FreeRTOS scheduler. The earlier P2 result is one fixed, non-vacuous generated-source witness. It remains useful as a regression witness, but it is not evidence for arbitrary task sets, priority assignments, ticks, delays, list populations, addresses, or source branches. The results counted here as universal have quantified statements that do not select that witness.

Status discipline. `Scheduler_Ordered_Insert_General_Loop`, `Scheduler_Ordered_Insert_General_Refinement` (the O4 work), and `Scheduler_Universal_Ordered_Insert_Composition` are **IN PROGRESS**. Their current source text is not counted as a checked theorem inventory entry. The composition theory is a draft without a session entry and has not been built.

Abstract

The proof development has moved beyond a single fixed P2 execution in several mathematically reusable directions. Isabelle now checks pure abstract delay semantics for arbitrary inputs, including modular 32-bit wake arithmetic, ready-list removal, delayed-list selection, frame facts, and a modular observation relation for the proof-port yield counter. Separate theorems show that delaying one task preserves every different ready task already present and identify the additional premise needed to conclude that some ready task remains.

At the representation layer, checker-green bridges connect generated scheduler list reads to the arbitrary-ring raw list model. A finite-task geometry theory treats an arbitrary finite live set, an injective guarded TCB map, pairwise-disjoint TCB regions, and explicit decoder laws; it derives separation and decoder round trips for whole TCBs and their embedded generic and event items. A separate finite-cardinality proof derives the formerly free `raw_count_can_increment` condition from `raw_xlist_rel` and insertion freshness, using the actual cardinality of the 32-bit pointer type.

The remaining hard path is source-level universal ordered insertion and its scheduler-wide composition. Trace-led analysis has identified the required generated-loop split invariant, the maximum-key bypass, the exact six-write alias classes, and a counterexample showing why freshness relative to one target list cannot establish a global scheduler frame. The generated loop proof, the general raw refinement, and the capacity-eliminating composition are still in progress. Consequently Gates L, H, and D remain open, and no whole-operation universal scheduler theorem is claimed.

1. Quantified acceptance boundary

The frozen build has `configMAX_PRIORITIES = 4`. Universality for this build means quantification over an arbitrary finite live-task set and an arbitrary priority function satisfying

$$\forall t \in \text{live}. \text{priority}(t) < 4.$$

It does not mean checking a finite table of examples. A qualifying Level-A `vTaskDelay` theorem must range over:

- every finite live-task set;
- every valid priority assignment into the four configured priorities;
- every well-formed population, order, and cursor of all ready rings;
- both physical delayed-list role assignments and arbitrary valid delayed rings;
- every admitted current task, 32-bit tick, and 32-bit delay argument;
- every generic decoder and heap layout satisfying the stated guard, injectivity, separation, and round-trip laws;
- every source scalar and proof-port observation admitted by the declared sequential API boundary;
- every source branch reachable from that boundary, including modular wrap and non-wrap cases.

The desired theorem must establish a non-vacuous generated-source execution, the complete concrete-to-abstract post-relation, invariant preservation at the declared phase, and a complete frame. Pure abstract equations, symbolic traces, isolated list transformers, or fixed witnesses are necessary components but are not substitutes for that theorem.

This acceptance boundary is configuration-specific. It does not quantify over arbitrary values of `configMAX_PRIORITIES`, and it does not include interrupt interference unless an explicit concurrency model is added.

2. Fixed P2 witness versus all-input results

The earlier P2 seal and the new results have different logical forms.

Evidence class	Quantification	What it establishes	What it does not establish
Fixed P2 witness	One artifact-specialised prestate and one selected generated-source path	Non-vacuity and correctness of that concrete path under the frozen translation and relation	Other task populations, priorities, delays, branches, heaps, or call sequences
Universal pure delay bricks	Arbitrary abstract state, tick/delay word, current-task option, task identity, and relevant rings	Exact abstract transition equations and preservation facts	Execution of generated C source or existence of a concrete preimage
Universal ABI/ring bridges	Arbitrary raw-list relation, cycle node, heap, and list pointer satisfying the bridge premises	Agreement of generated reads, raw reads, guards, keys, and successors	The generated loop's termination and poststate
Universal finite-task geometry	Arbitrary finite live set and decoder satisfying explicit storage and decoder laws	Cross-task separation, pointer inequality, and decoder round trips	Construction of such a decoder/heap for every reachable scheduler state
Derived count capacity	Arbitrary raw list and fresh item satisfying the legal input relations	The list count can increment without wrap	General ordered insertion or scheduler-wide ownership freshness

The fixed witness remains a regression test. It is deliberately excluded from the evidence used to justify the universal rows.

3. Checker-green universal abstract delay semantics

Let s be an arbitrary abstract scheduler state, d an arbitrary 32-bit delay word, and t the current task when one exists. For a positive delay the definition

$$\begin{aligned} & \text{positive_delay_state}(d, t, s) \\ &= \\ & \text{request_yield} \\ & (\text{block_task_at}(\text{tick}(s) + _ (32)d, t, s)) \end{aligned}$$

uses word addition, not natural-number addition. The delayed destination is selected by the source-shaped unsigned comparison

$$\text{tick}(s) + _ (32)d < \text{tick}(s).$$

The checker-green theorem `task_delay_abs_positive_universal_effects` proves, without fixing an input state:

- the current task's wake field becomes the modular sum;
- every other task's wake field is unchanged;
- the current generic item is removed from its priority's ready ring;
- every other priority's ready ring is unchanged;
- the current or overflow delayed ring receives an ordered insertion according to the modular comparison;
- the delayed ring not selected by the comparison is unchanged;
- the abstract current task and tick are unchanged at this phase;
- the abstract yield count is incremented once.

The theorem `task_delay_abs_all_inputs_capstone` covers all abstract delay inputs by splitting only on `ticks = 0` and the current-task option. It is a complete characterization of the current **pure abstract function**. It is not a generated-source refinement theorem.

The generated proof-port operation has a separate modular observation theorem. The relation

$$\begin{aligned} & \text{yield_count_mod_rel}(w, n) \\ & \Leftrightarrow \\ & w = \text{of_nat}(n) \end{aligned}$$

relates the 32-bit concrete counter to the low 32 bits of the unbounded abstract event count.

`eal6_port_yield_mod_refines_request_yield` checks the generated yield operation against `request_yield` without a no-wrap assumption. This closes the modular yield observation itself, not the positive-delay source path around it.

4. Ready-task preservation and the necessary premise

The statement that delaying the current task preserves every ready member is false: if it is the only member of its ready ring, removal empties that ring. The checker-green lemma `remove_only_ready_member_counterexample` records this failure at the abstract list level.

The correct universal statement is `task_delay_abs_preserves_other_ready_member`:

$$\begin{aligned} & \text{current}(s) = \text{Some}(c), \\ & G(u) \in \text{ready}(s, p), \\ & u \neq c \\ & \Rightarrow \\ & G(u) \in \text{ready}(\text{task_delay_abs}(d, s), p). \end{aligned}$$

Thus some ready task remains only when a different ready task is supplied. `task_delay_abs_some_ready_task_remains` makes that premise explicit and retains the configured priority bound. The idle-task corollaries are instances of the general theorem, not evidence based on one fixed scheduler state.

5. Generic generated/raw ABI and ring bridges

The generated scheduler and the raw list development use different record universes. `scheduler_item_of_raw` is an address-preserving coercion between their item-pointer types. The checker-green bridge layer proves, for an arbitrary raw list relation and arbitrary node in its cycle:

- the generated pointer and its raw coercion have the same machine address;
- the cycle node and its raw successor satisfy the generated `c_guard` obligations;
- the generated `pxNext` read equals the raw successor;
- the generated item-key read equals `raw_key_at`;
- for live ring nodes, that raw key equals the abstract `item_key`;
- the generated while guard is equivalent to the corresponding comparison of raw keys.

The packaged theorem is `scheduler_ordered_loop_read_bridge`. Its statement is generic in the heap, list, ring, and cycle node. The theory imports artifact-generated ABI definitions through P2-named sessions, but its theorem does not fix a P2 task, address, ring length, key, or branch.

These read bridges remove a representation mismatch. They do not by themselves prove that the generated loop maintains an invariant or reaches the correct exit.

6. Universal finite-task geometry and decoder laws

For a live set L and decoder D , `universal_tcb_geometry L D` requires:

```
finite(L),
inj_on(D_tcb_ptr, L),
∀ t ∈ L. guard(D_tcb_ptr(t)),
∀ t, u ∈ L. t ≠ u ⇒
Reg(TCB)(t) ∩ Reg(TCB)(u) = ∅.
```

The component type ranges over the whole TCB, the embedded generic item, and the embedded event item. `universal_different_live_component_regions_disjoint` proves that every component of one live task is disjoint from every component of a different live task. `universal_different_live_component_bases_neq` derives the corresponding base-pointer inequality. No task count, task identity, or concrete address is fixed.

Storage geometry alone is not enough to justify decoding. `universal_decoder_laws` separately states forward and inverse laws for TCB, generic-item, and event-item decoding, and requires every successfully decoded node owner to be live. The theorem `pointer_injectivity_does_not_force_decoder_roundtrip` supplies a counterexample to the tempting inference from pointer injectivity alone. The three decoder iff lemmas and `universal_geometry_scheduler_decode_rel` transfer the explicit laws into the existing scheduler decoder relation.

This is a conditional generic geometry theorem: later work must still derive its premises from a complete generic scheduler representation and show that the representation is inhabited where claimed.

7. Checker-green derivation of list-count capacity

The raw ordered-insert development previously carried

```
raw_count_can_increment xs
```

as a free premise. Gate-L capacity work now derives it from the legal list input.

From `raw_xlist_rel h lp xs`, Isabelle obtains

$$\text{distinct}(E \setminus \# \text{ring}(xs)),$$

where E is the list sentinel. From `raw_fresh_for_insert lp (ring xs) p`, it obtains $p \neq E$ and $p \notin \text{set}(\text{ring}(xs))$. Hence

$$\text{distinct}(p \setminus \# E \setminus \# \text{ring}(xs)).$$

The proof does not assume an address bound. It proves the pointer-cardinality bridge

$$\text{CARD}(\text{raw_node_id}) = 2^{(32)}$$

from the `Ptr` constructor, the 32-bit address type, `card_image`, and `card_word`. Finite-set cardinality then gives

$$|\text{ring}(xs)| + 2 \leq 2^{(32)},$$

and therefore

$$\begin{aligned} |\text{ring}(xs)| &< 2^{(32)} - 1 \\ &= \text{unat}(\text{max_word} : 32 \setminus \text{word}). \end{aligned}$$

The checker-green theorem is:

```
theorem raw_xlist_rel_fresh_count_can_increment:
  assumes "raw_xlist_rel h lp xs"
    and "raw_fresh_for_insert lp (ring xs) p"
  shows "raw_count_can_increment xs"
```

There is no hidden length constant, fixed address, or retained capacity premise.

8. Trace-led generated-loop invariant - design fixed, proof in progress

The trace audit is invariant-design evidence, not proof evidence. Let:

- E be the sentinel;
- N be the fresh item;
- k be the key read from N ;
- $xs=P@S$ be a ghost split of the old ring;
- $c=last(E\#P)$ be the generated iterator's predecessor;
- $q=hd(S@[E])$ be the raw successor.

For the ordinary branch $k \neq MAX$, the proposed mutable invariant carries:

```
generated state and heap = entry state and heap
xs = P @ S
every key in P is <= k
iterator address = last (E # P)
```

The derived view gives $q=hd(S@[E])$. At a continuing head $S=y\#ys$, the ghost transition is

```
(P,y\#ys,c)
longmapsto
(P@[y],ys,y),
```

with strict variant decrease

$$|ys| < |y\#ys|.$$

The empty suffix exposes why the sentinel key is essential. Without the checked fact that the sentinel key is MAX , a weak invariant could permit a true guard while $S=[]$, leaving both iterator and $|S|$ unchanged. The invariant must derive the sentinel fact from the entry relation; it may not assume the desired exit.

The maximum-key source branch bypasses the loop. Since every 32-bit key is at most MAX , it must join the ordinary exit description with the complete prefix and empty suffix:

```
P=xs, S=[],
c=last(E\#xs), q=E.
```

The pure split, prefix, suffix, and maximum-key facts in `Scheduler_Ordered_Insert_General_Source` are checked foundations. The generated-carrier loop session and its advertised non-max, max, and joined exact-state theorems remain **IN PROGRESS** and are not counted as proved.

9. Six writes and legal alias classes

After scan exit the source performs exactly these writes in this order:

Step	Target	Required value
1	<code>N.next</code>	q
2	<code>q.previous</code>	N
3	<code>N.previous</code>	c
4	<code>c.next</code>	N
5	<code>N.container</code>	target list L
6	<code>L.count</code>	old count plus one

A correct proof must preserve the following legal alias classes rather than ruling them out globally:

- empty ring: $c=q=E$;
- insertion before the first old item: $c=E$;
- insertion after the last old item: $q=E$;
- strict middle insertion: c and q are ordinary adjacent nodes;
- sentinel link fields and `L.count` inhabit the same packed list object;
- the three writes to fields of N inhabit the same item object.

Freshness must still prove $N \neq c$, $N \neq q$, and separation of the entire writable region of N from the target root and every old target-list item. The exact byte frame is outside the union

```
N.next, q.previous, N.previous, c.next, N.container, L.count.
```

The O4 refinement is responsible for connecting these field-level facts to the raw ordered-list relation. That session is **IN PROGRESS**; declarations present in its current theory file are not accepted as checker-green inventory entries.

10. Local spatial freshness is not global ownership freshness

`raw_fresh_for_insert L xs N` is a local target-list predicate. It says that N is distinct from the sentinel and old ring of L and spatially separated from their relevant regions. This is enough for a local target-list transformer.

It is not enough for a scheduler-wide frame. Consider two valid roots L and M . Let L be empty and let N be the sole member of M . Then N can satisfy spatial freshness relative to L , yet inserting it into L overwrites the links and container by which M represents N . The local postcondition may hold while the global scheduler representation is destroyed.

Therefore Gate H needs a separate remove-to-insert intermediate invariant, for example

`N.container=NULL`

under a global invariant that makes the container field faithful to list membership, or an equivalent theorem that N is globally unlinked from every scheduler-owned root. This fact must be **derived from the preceding removal**. It must not be added as a postcondition-shaped premise to the local ordered-insert theorem.

The drafted capacity-eliminating composition remains a local list refinement and does not claim this global frame.

11. Open gates

Gate L - universal list layer: open

Checker-green sub-obligations include the arbitrary-ring read bridges, pure scan/split facts, existing general removal and fresh insert-end bricks, and the derived count-capacity theorem. Gate L remains open because:

- the generated-loop carrier proof is **IN PROGRESS**;
- the all-ring raw ordered-insert refinement O4 is **IN PROGRESS**;
- the exact six-write alias/frame proof has not entered the accepted inventory;
- the capacity-eliminating local composition is an unbuilt draft.

Gate H - universal heap/state vTaskDelay: open

Checker-green sub-obligations include all-input pure delay equations, modular yield observation, preservation of a distinct ready task, and conditional finite-task geometry/decoder laws. Gate H remains open because there is no checker-green generated-source theorem for arbitrary positive delays and arbitrary scheduler states. Still required are:

- one complete generic scheduler relation over all relevant roots, TCB fields, scalars, cursors, and delayed roles;
- generic preimage or inhabitation evidence for the claimed state class;
- the remove-to-insert globally-unlinked ownership fact;
- cross-root and unrelated-TCB frame theorems;
- source-level removal, ordered insertion, suspend/resume, branch, guard, and termination composition;
- an audited capstone whose assumptions contain the declared API boundary and no selected poststate.

Gate D - five-root dispatcher and trace gate: open

No checker-green universal dispatcher or arbitrary finite-call trace theorem exists. The remaining obligations include whole-operation refinements for all five roots, their reachable helper closure and branches, preservation of one common invariant, and sequential composition over arbitrary finite valid call traces. Concurrency, port context switching, lifecycle operations, allocator correctness, compiler correctness, and machine-code refinement remain separate stronger layers unless explicitly added.

12. Tool and trust boundary

The accepted logical evidence is ordinary Isabelle theorem checking with `quick_and_dirty = false`. The listed checker-green theories contain no accepted `sorry`, `oops`, admitted desired axiom, or oracle shortcut.

The trust and evidence boundary is:

```

frozen source/configuration and externally validated artifact metadata
    |
    v
CPParser / AutoCorres2 translation
    |
    v
generated monadic source semantics and ABI records
    |
    v
Isabelle/HOL definitions and proofs
    |
    v
Isabelle kernel theorem objects

```

The builder, artifact ledger, CParser, AutoCorres2, and Isabelle implementation remain toolchain assumptions. The correspondence from the frozen binary/layout evidence to generated address definitions is externally validated; it is not an Isabelle theorem. The universal abstract theories do not repair that artifact link, and abstract execution equations do not imply generated-source execution.

ChatGPT Pro contributes design review and adversarial questions, not theorem objects. Symbolic traces, executable probes, or an SMT backend may accelerate invariant discovery, counterexample search, and verification-condition simplification, but their output is untrusted. A fact enters the proof inventory only when replayed as an ordinary theorem in the declared Isabelle session.

13. Theorem ledger

Status	Theory / theorem	Quantified content	Boundary
CHECKER-GREEN	eal6_port_yield_mod_refines_request_yield	Generated proof-port yield refines modular abstract request_yield	Yield primitive only
CHECKER-GREEN	task_delay_abs_positive_universal_effects	Arbitrary positive delay, state, current task, modular branch, ready/delayed effects	Pure abstract semantics
CHECKER-GREEN	task_delay_abs_all_inputs_capstone	All abstract delay words and current-task options	Pure abstract semantics
CHECKER-GREEN	task_delay_abs_preserves_other_ready_member	Preservation of every different ready task	Requires an existing distinct ready member
CHECKER-GREEN	task_delay_abs_some_ready_task_remains	Existence of a post-delay ready task under an explicit distinct-member premise	Does not create a ready task from no premise
CHECKER-GREEN	scheduler_ordered_loop_read_bridge	Generated/raw next, key, guard, and cycle-node agreement for arbitrary valid rings	Read bridge, not loop correctness
CHECKER-GREEN	universal_different_live_component_regions_disjoint	Cross-task separation for TCB/generic/event components over arbitrary finite live sets	Conditional on explicit geometry
CHECKER-GREEN	universal_different_live_component_bases_neq	Cross-task component pointer inequality	Conditional on explicit geometry
CHECKER-GREEN	decoder iff lemmas and universal_geometry_scheduler_decode_rel	Forward/inverse decoder facts and transfer to existing relation	Conditional on explicit decoder laws
CHECKER-GREEN	raw_node_id_card	Actual raw pointer type has cardinality $2^{(32)}$	Frozen 32-bit address model
CHECKER-GREEN	raw_xlist_rel_fresh_count_can_increment	Legal raw list plus fresh item implies count capacity	Local list input
CHECKER-GREEN FOUNDATION	Scheduler_Ordered_Insert_General_Source scan/split lemmas	Pure P/S split, prefix/suffix, predecessor/successor, and maximum-key facts	Does not establish generated loop theorem
IN PROGRESS	Scheduler_Ordered_Insert_General_Loop	Generated carrier invariant, non-max loop, max bypass, exact joined state	Not accepted yet
IN PROGRESS	Scheduler_Ordered_Insert_General_Refinement / O4	Six-write raw transformer, exact effect, relation and frame	Not accepted yet
IN PROGRESS	Scheduler_Universal_Ordered_Insert_Composition	Remove free capacity premise from local transformer interface	Draft; no ROOT entry; unbuilt
FIXED WITNESS ONLY	frozen P2 capstone	One inhabited generated-source refinement path	Regression evidence, not universal evidence

14. Current conclusion

The development now contains genuine universal-input mathematics: arbitrary abstract delay arithmetic and effects, a modular generated yield observation, ready-member preservation, arbitrary-ring generated/raw read bridges, finite-task component geometry with explicit decoder laws, and a derived 32-bit count-capacity theorem. None of these relies on enumerating priority combinations or fixing a semantic witness.

The central source-level theorem is nevertheless still missing. Until the generated ordered-insert loop, O4 raw refinement, local composition, global ownership-fresh remove-to-insert bridge, and complete generated vTaskDelay composition are checker-green, Gate L and Gate H remain open. Gate D remains open independently. The correct current result is therefore:

Reusable universal-input proof bricks are checker-green; universal FreeRTOS V6.1.1 scheduler functional correctness is not yet proved.